CMPSC 461: Programming Language Concepts
Assignment 4

Name: Mohammad Abu Shariha
User ID: mma7020

October 17, 2025

Problem 1: Parameter Passing Exercise

Initial global variables: $x = 3$, $y = 2$, $z = 5$.

(a) update(x, y) - Pass by Value

Concept: Local copies a and b are created. a is initialized to x (3) and b is initialized to y (2). Assignments to a or b are local; assignments to x, y, or z change the globals.

Trace:

- **Initial:** Globals: $x = 3$, $y = 2$, $z = 5$. Locals: $a = 3$, $b = 2$.
- **L1 ($a = a + y$):** $a = 3 + 2 = 5$.
- **L2 ($y = b + x$):** $y = 2 + 3 = 5$. (Global y is now 5)
- **L3 ($b = a - z$):** $b = 5 - 5 = 0$.
- **L4 ($x = b + y + a$):** $x = 0 + 5 + 5 = 10$. (Global x is now 10)
- **L5 ($z = x + a - b$):** $z = 10 + 5 - 0 = 15$. (Global z is now 15)

Final Globals: $x = 10$, $y = 5$, $z = 15$.

(b) update(x, z) - Pass by Reference

Concept: a becomes an alias for x ($a \leftrightarrow x$), and b becomes an alias for z ($b \leftrightarrow z$).

Trace:

- **Initial:** Globals: $x = 3$, $y = 2$, $z = 5$.
- **L1 ($a = a + y$):** $x = x + y \Rightarrow x = 3 + 2 = 5$.
- **L2 ($y = b + x$):** $y = z + x \Rightarrow y = 5 + 5 = 10$.

- **L3** ($b = a - z$)**:** $z = x - z \Rightarrow z = 5 - 5 = 0$.
- **L4** ($x = b + y + a$)**:** $x = z + y + x \Rightarrow x = 0 + 10 + 5 = 15$.
- **L5** ($z = x + a - b$)**:** $z = x + x - z \Rightarrow z = 15 + 15 - 0 = 30$.

Final Globals: $x = 15$, $y = 10$, $z = 30$.

(c) update(y, z) - Pass by Value-Result

Concept: Copy-in: local $a = y = 2$, $b = z = 5$. Execute. Copy-out: $y = a$, $z = b$.

Trace (Execution):

- **Initial:** Globals: $x = 3$, $y = 2$, $z = 5$. Locals: $a = 2$, $b = 5$.
- **L1** ($a = a + y$)**:** $a = 2 + 2 = 4$.
- **L2** ($y = b + x$)**:** $y = 5 + 3 = 8$. (Global y is now 8)
- **L3** ($b = a - z$)**:** $b = 4 - 5 = -1$.
- **L4** ($x = b + y + a$)**:** $x = -1 + 8 + 4 = 11$. (Global x is now 11)
- **L5** ($z = x + a - b$)**:** $z = 11 + 4 - (-1) = 16$. (Global z is now 16)

Trace (Copy-Out):

- Copy $a \to y$: $y = 4$.
- Copy $b \to z$: $z = -1$.

Final Globals: $x = 11$, $y = 4$, $z = -1$.

(d) update(y, x) - Pass by Reference

Concept: a becomes an alias for y ($a \leftrightarrow y$), and b becomes an alias for x ($b \leftrightarrow x$).

Trace:

- **Initial:** Globals: $x = 3$, $y = 2$, $z = 5$.
- **L1** ($a = a + y$)**:** $y = y + y \Rightarrow y = 2 + 2 = 4$.
- **L2** ($y = b + x$)**:** $y = x + x \Rightarrow y = 3 + 3 = 6$.
- **L3** ($b = a - z$)**:** $x = y - z \Rightarrow x = 6 - 5 = 1$.
- **L4** ($x = b + y + a$)**:** $x = x + y + y \Rightarrow x = 1 + 6 + 6 = 13$.
- **L5** ($z = x + a - b$)**:** $z = x + y - x \Rightarrow z = 13 + 6 - 13 = 6$.

Final Globals: $x = 13$, $y = 6$, $z = 6$.

(2) Why Pass-by-Result is Not Applicable

Pass-by-result (or copy-out) is a scheme where formal parameters are uninitialized at the start of the function and their final values are copied back to the actual arguments upon return.

This scheme is not applicable here because the function `update` reads its formal parameters before they are assigned values:

- **Line 1 ($a = a + y$):** Reads the value of a on the right-hand side.
- **Line 2 ($y = b + x$):** Reads the value of b on the right-hand side.

If pass-by-result were used, a and b would be uninitialized at these lines, leading to undefined behavior.

Problem 2: Parameter Passing II

(1) Expansion, Output, and Why

Expansion: Macros perform pure textual substitution.

1. The call is `square(add(x,1))`.
2. The text `add(x,1)` is substituted for `x` in `x*x`.
3. This results in: `add(x,1) * add(x,1)`.
4. Next, `add(x,1)` is expanded to `x+1`.
5. The final expanded text is: `x+1 * x+1`.

Evaluation: The C compiler parses this expression according to operator precedence, where multiplication ($*$) is higher than addition ($+$).

- The expression is grouped as: `x + (1 * x) + 1`.
- With $x = 2$, this evaluates to: $2 + (1 * 2) + 1$.
- $2 + 2 + 1 = 5$.

Output: `5`

Why: The problem is caused by the lack of parentheses in the macro definitions, which leads to an incorrect order of operations when the macros are expanded.

(2) How to Fix the Issue

To prevent precedence errors, add parentheses around each parameter and around the entire macro body.

Corrected Macros:

```
#define add(a,b) ((a) + (b))
#define square(x) ((x) * (x))
```

With this fix, the expansion of `square(add(x,1))` becomes `((x+1) * (x+1))`. With $x = 2$, this evaluates to $((2 + 1) * (2 + 1))$, which is $3 * 3 = 9$.

Problem 3: Recursion

(1) Tail-Recursive Factorial

The original function is not tail-recursive because the multiplication by `n` happens *after* the recursive call returns. To make it tail-recursive, we use an accumulator (`acc`) to pass the intermediate product into the next recursive call.

```
def factorial_helper(n, acc):
    if n == 0:
        return acc  # Base case: return the accumulated product
    else:
        # Pass the new product (acc * n) into the next call
        return factorial_helper(n - 1, acc * n)

def factorial(n):
    # Start the helper with the initial accumulator value of 1
    return factorial_helper(n, 1)
```

(2) Non-Tail-Recursive Power

The given `power_tail` function is tail-recursive. To convert it to a non-tail-recursive form, we must perform an operation *after* the recursive call returns. Here, we perform the multiplication with `base` after the recursive call `power_non_tail(base, exp - 1)` completes.

```
def power_non_tail(base, exp):
    if exp == 0:
        return 1  # Base case
    else:
        # Multiplication happens *after* the recursive call returns
        return base * power_non_tail(base, exp - 1)
```

Problem 4: Exception Handling

(1) Output of Original Program

Trace:

1. `main` calls `A()`. `A` calls `B()`. `B` calls `C()`.
2. `C()` throws "Thrown in C".
3. `B()`'s `catch` block catches it.
4. **Prints:** `Caught in B: Thrown in C`
5. `B()` throws a new exception "From B".
6. `A()`'s `catch` block catches it.
7. **Prints:** `Caught in A: From B`
8. `A()` throws a new exception "From A".
9. `A()`'s `finally` block *must* execute before the exception propagates.
10. **Prints:** `Finally in A`
11. `main()`'s `catch` block catches "From A".
12. **Prints:** `Catch in main:  From A`

Final Output:

```
Caught in B: Thrown in C
Caught in A: From B
Finally in A
Catch in main: From A
```

(2) Modified Program

To achieve the desired output, the `main` method must be changed to call `B()` and `A()` in separate `try...catch` blocks. The `A` method must also be modified to throw and catch its own exception internally, without calling `B`.

Modified `main` Method:

```
public static void main(String[] args) {
    // First call: B()
    try {
        B();
    } catch (CustomException e) {
        // Catches the "From B" exception
        System.out.println("Catch in main: " + e.getMessage());
    }
```

```
    // Second call: A()
    try {
        A();
    } catch (CustomException e) {
        // Catches the "From A" exception
        System.out.println("Catch in main: " + e.getMessage());
    }
}
```

Modified `A` Method: (`B` and `C` remain unchanged)

```
public static void A() throws CustomException {
    try {
        // Throw the exception directly, instead of calling B()
        throw new CustomException("Direct exception from A");
    } catch (CustomException e) {
        // Catch the internal exception
        System.out.println("Caught in A: " + e.getMessage());
        // Rethrow the "From A" exception
        throw new CustomException("From A");
    } finally {
        // This still runs
        System.out.println("Finally in A");
    }
}
```

This modified code produces the exact required output.